OVERVIEW / □□

液态玻璃，是一条光学渲染链。

它不等于给 div 加 blur。CSS 定义外壳，Canvas 重建局部背景，WebGL 重新计算折射与光，生命周期让玻璃在滚动、换肤和长聊天里保持正确。

01

SCENE

背景 / 主题 / 页面状态

02

SAMPLE

同一坐标系里的局部背景

03

OPTICS

纹理 / uniforms / 材质

04

UI

文字与交互独立叠加

RENDER CHAIN

SCENE

壁纸与页面底色

CROP

按组件位置采样

FIELD

圆角折射场

MATERIAL

透明 / 烟灰

SNAPSHOT

二维稳定输出

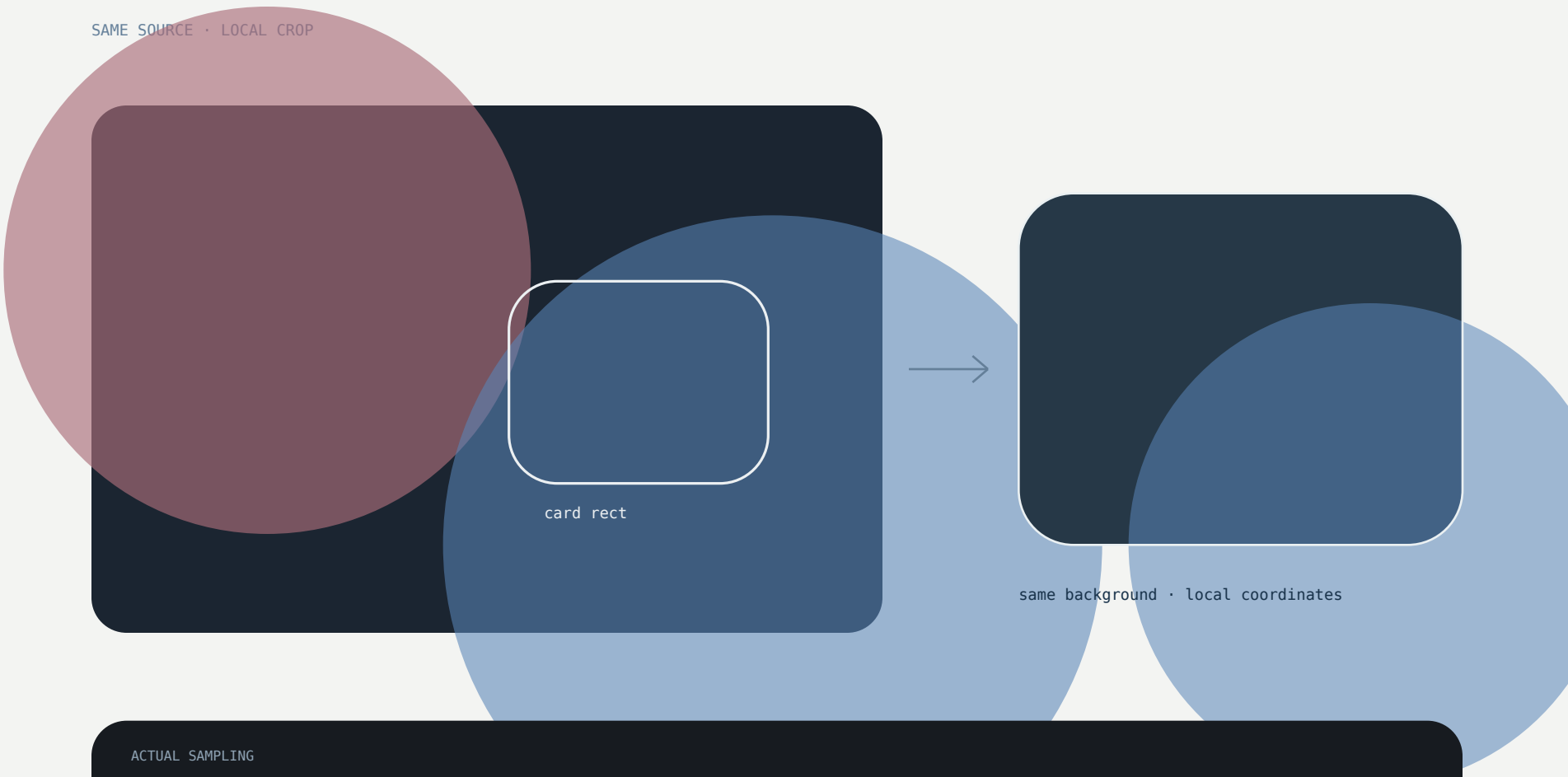
□□□□□

玻璃先看见背景，再把它折射；文字始终留在上层。

01 / BACKGROUND SAMPLING

先对齐背景，玻璃才不会穿帮。

每个组件都从同一张全屏背景里，按自己在视口中的位置切出对应区域。我们重建壁纸、页面底色与主题蒙层；不会实时截图任意 DOM。



ACTUAL SAMPLING

```
const rect = card.getBoundingClientRect();
const scale = Math.max(vw / iw, vh / ih);
const dx = (vw - iw*scale)/2 - rect.left;
const dy = (vh - ih*scale)/2 - rect.top;
ctx.drawImage(image, dx, dy, iw*scale, ih*scale);
gl.texImage2D(..., backdrop);
```

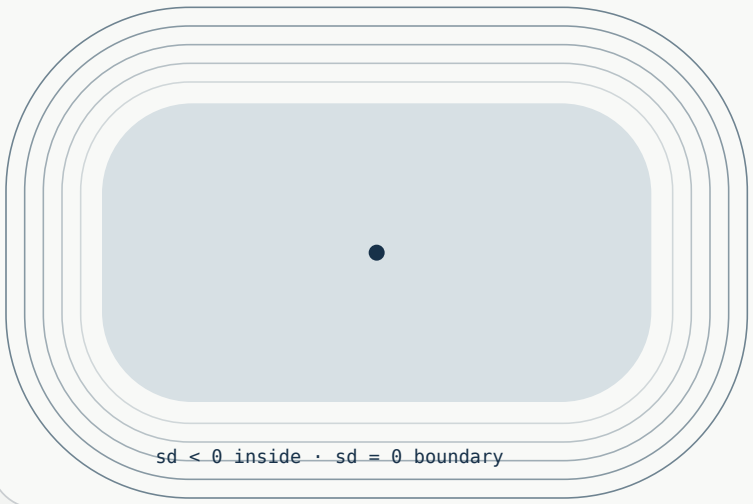
关键限制：普通文字、视频与叠在玻璃后的任意 DOM 不会自动进入纹理；多层玻璃也不是递归折射。公开 demo 使用同一张自托管背景来保证坐标一致。

02 / ROUNDED GEOMETRY

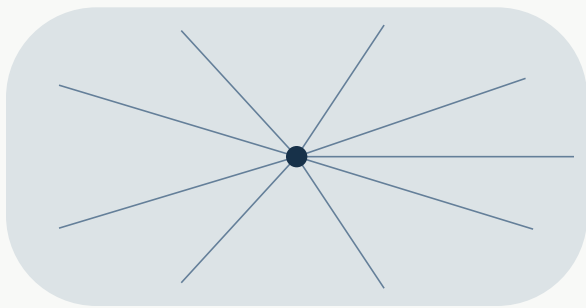
形状不是裁出来的，光也要认识圆角。

CSS 的尺寸与圆角会作为 uniforms 传进 Shader。roundBox() 给出真实圆角矩形的有符号距离；radialBoundary() 则让任意长宽比都拥有从中心到边缘的统一光学坐标。

SIGNED DISTANCE



NORMALIZED FIELD



ROUNDED-RECT SDF

```
float roundBox(vec2 point) {
  vec2 q = abs(point-u_res*.5)
    -(u_res*.5-vec2(u_corner));
  return length(max(q,0.))
    +min(max(q.x,q.y),0.)-u_corner;
}
```

FIELD COORDINATES

```
vec2 rel = px - u_res*.5;
vec2 p = rel / radialBoundary(rel);
float rr = clamp(length(p), 0., 1.);

// center ≈ 0 · edge ≈ 1
```

这一步解决了“宽卡片被当成拉长的圆形透镜”的问题。后面的折射、暗边和高光都共享同一组几何坐标。

03 / REFRACTION FIELD

折射覆盖整块表面，而不是一圈边框。

径向透镜负责整体的厚度变化，低频双轴波动提供轻微流动。两者相加后转换成 UV 偏移，再从背景纹理的另一个位置取色。

FIELD COMPOSITION

p	rr	wave	shift	refrUv
归一化位置	中心到边缘	双轴低频扰动	透镜 + 波动	重新采样背景

CONTINUOUS INTERIOR LENS

```
float edgeGain = smoothstep(0.05, 0.97, rr);
float strength = (0.052 + 0.12*edgeGain)
  * u_refract * u_lDistort;
vec2 wave = vec2(
  sin(p.y*3.1+p.x*1.4+u_time*.18)*cos(p.x*2.2),
  cos(p.x*3.4-p.y*1.3-u_time*.15)*sin(p.y*2.1+.8)
);
vec2 shift = p*lensShift + wave*strength;
vec2 refrUv = v_uv + vec2(shift.x/u_res.x,
                          -shift.y/u_res.y);
vec3 refr = texture2D(u_bg, clamp(refrUv,.001,.999)).rgb;
```

TUNING BASELINE

26	0	0	25	13	15
refract	tint	blur	edge	shine	wobble

参数是这套 Shader 的调节刻度，并非物理单位。位移太强会像水波，过度集中在边缘则会重新退化成“玻璃边框”。

04 / MATERIAL & LIGHT

同一套光学，两种透射性格。

透明与烟灰材质共用 Shader。区别来自透射衰减、内侧吸光、清透折射带，以及随背景亮度变化的边缘配方。





LUMINANCE ADAPTATION

```
lum = dot(color, vec3(0.2126, 0.7152, 0.0722))
```

亮场：暗边更窄、更轻，均匀白环被抑制。 深场：保留更多吸光厚度与局部高光。

ADAPTIVE SMOKE + DIRECTIONAL GLINT

```
01    float bright = smoothstep(0.42, 0.84, lum);
02    float smokeWidth = mix(7.2, 4.25, bright);
03    float edgeLoss  = mix(0.27, 0.085, bright);
04
05    float hairline  = exp(-pow((sd + .46)/.43, 2.));
06    float upperLeft = exp(-pow((ang + 2.38)/.60, 2.));
07    float lowerRight= exp(-pow((ang - .76)/.62, 2.));
08    float glint = hairline*(upperLeft*.66 + lowerRight*.48);
```

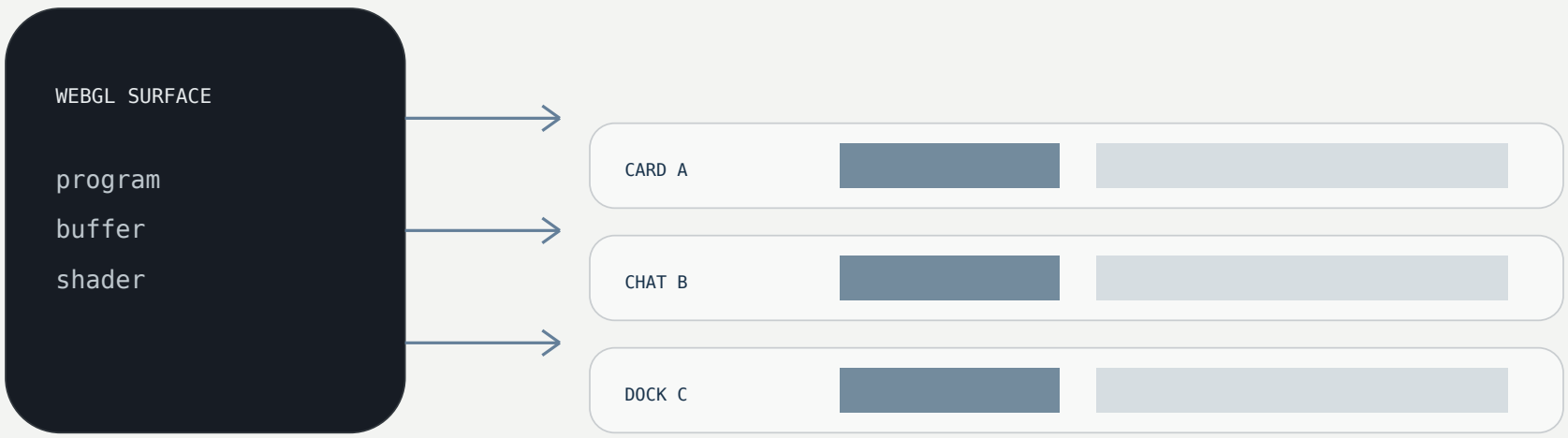
方向性高光比整圈白描边更重要。浅色背景尤其需要收掉全周 Fresnel，否则玻璃会变成发亮的塑料按钮。

05 / SHARED RENDERER

一套 WebGL，许多稳定的玻璃快照。

长聊天不能为每个气泡申请一个 WebGL 上下文。当前版本由整页共享 program 与 buffer；每个组件只保存自己的背景纹理和二维输出 Canvas。

SHARED GPU · INDEPENDENT OUTPUTS



RENDER THEN SNAPSHOT

```
    gl.drawArrays(gl.TRIANGLE_STRIP, 0, 4);

    // Copy before the shared drawing buffer is discarded.
    output.clearRect(0, 0, width, height);
    output.drawImage(renderSurface, 0, 0);

    // Text remains normal DOM above the canvas.
```

WHY IT MATTERS

屏幕外的气泡可以释放 GPU 纹理，却保留已经渲染好的二维画面；WebGL context 暂时丢失时，旧内容也不会全部变空。

这是 v4 相比旧方案最关键的工程变化：从“每块玻璃一个上下文”，转为“一个渲染器，多份稳定输出”。

06 / LIFECYCLE & REUSE

玻璃一直活着，但只在必要时工作。

背景更新与材质动画分开。滚动、尺寸、主题和壁纸变化只标记 dirty；可见组件按上限频率绘制，离屏后释放纹理并保留二维快照。

LIFECYCLE

RESIZE	重读尺寸与圆角
SCROLL	重算局部采样位置
THEME / BG	重建背景纹理
OFFSCREEN	暂停并释放 texture
DISPOSE	清理 observer 与资源

FRAME BUDGET

```
if (!visible || document.hidden) return;
if (!reduced && ms-last < 48) return;
last = ms;

if (dirty) redrawTexture();
draw(reduced ? 0 : ms * 0.001);
```

CURRENT BUDGETS

≤ 1.5	≈ 20	1	2.25 px	CSS blur
DPR	VISIBLE FPS	WEBGL CONTEXT	HERO BLUR	FALLBACK

PUBLIC SCOPE

公开仓库只提供光学思路、最小 demo 与技术册。真实 Home、私人背景、聊天逻辑和个人数据不在其中。

UPSTREAM: VII-Cae/hyalite--liquid-glass (MIT) · everett745/liquid-glass (NO LICENCE DECLARED)

PROJECT: github.com/yanceydaisy/liquid-glass-note